



Testes de Mutação

Teste de Mutação

- O teste de mutação é uma técnica que foca na medição da adequação dos casos de testes.
- O objetivo é encontrar as fraquezas nos casos de teste.
- No teste de mutação alguns erros são introduzidos nos programas através de pequenos desvios sintáticos que alteram a semântica do programa e, conseqüentemente, conduzem o programa a um comportamento incorreto.

Teste de Mutação

- O teste de mutação não é uma estratégia de teste como fluxo de controle ou fluxo de dados. É usado para complementar as técnicas tradicionais.

Teste de Mutação

- Uma **mutação** de um programa é uma modificação criada pela introdução de uma pequena mudança sintática, que não provoca nenhum erro sintático, no seu código.
- Um programa modificado é chamado de **mutante**.
- Alguns mutantes são equivalentes ao programa original, enquanto outros são defeituosos (**faulty**).
- Um mutante é dito morto (**killed**) quando a execução de um caso de teste causa uma falha e o mutante é considerado **dead**.
- Mutantes vivos são aqueles que não são mortos.

Exemplo de Teste de Mutaç o [Naik]

Considere o seguinte c digo que encontra a posi  o do primeiro maior valor que aparece no array:

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.     for i = 2 to 3 do
6.         if (atoi(argv[i]) > atoi(argv[r])) r = i;
7. printf("Value of the rank is %d \n", r);
8. exit(0); }
```

Exemplo de Teste de Mutação

Considere os seguintes casos de testes:

Caso de Teste 1: Entrada: 1 2 3 Saída esperada: posição = 3

Caso de Teste 2: Entrada: 1 2 1 Saída esperada: posição = 2

Caso de Teste 3: Entrada: 3 1 2 Saída esperada: posição = 1

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.   for i = 2 to 3 do
6.     if (atoi(argv[i]) > atoi(argv[r])) r = i;
7.   printf("Value of the rank is %d \n", r);
8.   exit(0); }
```

Exemplo de Teste de Mutação

Considere os seguintes mutantes:

Mutante 1: Mudar a **linha 5** de: `for i = 2 to 3 do`
para: **`for i = 1 to 3 do`**

Mutante 2: Mudar a **linha 6** de: `if (atoi(argv[i]) > atoi(argv[r])) r = i;`
para: **`if (i > atoi(argv[r])) r = i;`**

Mutante 3: Mudar a **linha 6** de: `if (atoi(argv[i]) > atoi(argv[r])) r = i;`
para: **`if (atoi(argv[i]) >= atoi(argv[r])) r = i;`**

Mutante 4: Mudar a **linha 6** de: `if (atoi(argv[i]) > atoi(argv[r])) r = i;`
para: **`if (atoi(argv[r]) > atoi(argv[r])) r = i;`**

Exemplo de Teste de Mutação

Mutante 1: Mudar a linha 5 para

for i = 1 to 3 do

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.     for i = 2 to 3 do for i = 1 to 3 do
6.         if (atoi(argv[i]) > atoi(argv[r])) r = i;
7. printf("Value of the rank is %d \n", r);
8. exit(0); }
```

- O programa modificado passa nos testes. O mutante 1 não é morto, ou seja, continua vivo.

Caso de Teste 1:	<u>Entrada:</u> 1 2 3	<u>Saída esperada:</u> posição = 3
Caso de Teste 2:	<u>Entrada:</u> 1 2 1	<u>Saída esperada:</u> posição = 2
Caso de Teste 3:	<u>Entrada:</u> 3 1 2	<u>Saída esperada:</u> posição = 1

Exemplo de Teste de Mutação

Mutante 2: Mudar a **linha 6** para

`if (i > atoi(argv[r])) r = i;`

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.   for i = 2 to 3 do if (i > atoi(argv[r])) r = i;
6.   if (atoi(argv[i]) > atoi(argv[r])) r = i;
7.   printf("Value of the rank is %d \n", r);
8.   exit(0); }
```

- O programa modificado falha no caso de teste 2. O mutante 2 é morto.

Caso de Teste 1:	<u>Entrada:</u> 1 2 3	<u>Saída esperada:</u> posição = 3
Caso de Teste 2:	<u>Entrada:</u> 1 2 1	<u>Saída esperada:</u> posição = 2 3
Caso de Teste 3:	<u>Entrada:</u> 3 1 2	<u>Saída esperada:</u> posição = 1

Exemplo de Teste de Mutação

Mutante 3: Mudar a **linha 6** para

`if (atoi(argv[i]) >= atoi(argv[r])) r = i;`

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.   for i = 2 to 3 do if (atoi(argv[i]) >= atoi(argv[r])) r = i;
6.   if (atoi(argv[i]) > atoi(argv[r])) r = i;
7. printf("Value of the rank is %d \n", r);
8. exit(0); }
```

- O programa modificado passa nos testes. O mutante 3 não é morto, ou seja, continua vivo.

Caso de Teste 1:	<u>Entrada:</u> 1 2 3	<u>Saída esperada:</u> posição = 3
Caso de Teste 2:	<u>Entrada:</u> 1 2 1	<u>Saída esperada:</u> posição = 2
Caso de Teste 3:	<u>Entrada:</u> 3 1 2	<u>Saída esperada:</u> posição = 1

Exemplo de Teste de Mutação

Mutante 4: Mudar a **linha 6** para

`if (atoi(argv[r]) > atoi(argv[r])) r = i;`

```
1. main(argc,argv)
2. int argc, r, i;
3. char *argv[];
4. { r = 1;
5.   for i = 2 to 3 do if (atoi(argv[r]) > atoi(argv[r])) r = i;
6.   if (atoi(argv[i]) > atoi(argv[r])) r = i;
7.   printf("Value of the rank is %d \n", r);
8.   exit(0); }
```

- O programa modificado falha nos casos de teste 1 e 2. O mutante 4 é morto.

Caso de Teste 1:	<u>Entrada:</u> 1 2 3	<u>Saída esperada:</u> posição = 3 1
Caso de Teste 2:	<u>Entrada:</u> 1 2 1	<u>Saída esperada:</u> posição = 2 1
Caso de Teste 3:	<u>Entrada:</u> 3 1 2	<u>Saída esperada:</u> posição = 1

Exemplo de Teste de Mutação

Resultado:

- 4 mutantes criados e 2 mutantes mortos → 50%

Os 2 mutantes que não foram mortos (mutantes vivos) precisam ser analisados para verificar se são equivalentes ou se poderiam ter sido mortos.

Exemplo de Teste de Mutação

Considerando que:

- ❑ O mutante 1 é equivalente ao programa original (no mutante 1 a variável i começa com 1 e no programa original começa com 2, não influenciando no resultado da variável r).
- ❑ É adicionado um caso de teste 4 com entrada: 2 2 1 e saída 1, através do qual o mutante 3 é morto.

O resultado seria, então:

- 4 mutantes criados, 3 mutantes mortos e 1 equivalente → 100%

Exemplos de Mutações

- Retira um comando de cada vez do programa.
- Substitui um operador relacional por outro operador relacional.
- Substitui uma variável escalar pelo seu valor sucessor e predecessor.
- Substitui variáveis escalares por constantes.
- Substitui o comando while por do-while.
- Interrompe a execução do laço após duas execuções.
- Substitui uma constante por outra constante.
- Força cada variável escalar a possuir cada um dos valores: negativo, positivo e zero.

Conceitos

Mutante P_i e Programa P equivalentes: os seus comportamentos não são diferenciados por nenhum conjunto de casos de teste.

Mutante P_i killable (pode ser morto): os casos de testes são insuficientes para matar o mutante P_i . Novos casos de teste precisam ser criados.

- É preciso ressaltar que, em geral, a equivalência entre programas é uma questão indecidível e requer a intervenção do testador.

Processo do Teste de Mutação

Passos a serem seguidos na aplicação de testes de mutação:

- **Passo 1:** Considere um programa P e um conjunto de casos de testes T .
- **Passo 2:** Execute cada caso de teste T . Se o teste falhar o programa deve ser modificado e testado novamente.
- **Passo 3:** Crie um conjunto de mutantes $\{P_i\}$, cada um com uma pequena modificação de P .

Processo do Teste de Mutação

- **Passo 4:** Execute cada caso de teste com cada mutante P_i .

Se a saída é diferente da saída original, o mutante é considerado incorreto e é dito ser morto pelo caso de teste.

Se a saída produz o mesmo resultado da saída original, então:

- Ou eles são equivalentes
- Ou P_i pode ser morto (killable).

Processo do Teste de Mutação

- **Passo 5:** Calcule o escore da mutação para o conjunto de casos de testes T que relaciona o número de mutantes mortos com o número de mutantes gerados.

$$\text{Escore de Mutação} = 100 \times D / (N - E)$$

- D = número de mutantes mortos
- N = número total de mutantes
- E = número de mutantes equivalentes

Teste de Mutação

- **Passo 6:** Se a adequação de mutação não for suficientemente alta, então projete um novo caso de teste que distingue P_i e P_j e o adicione ao conjunto T e volte ao passo 2.

Teste de Mutação

- Um problema relacionado com o teste de mutação é o seu alto custo.
 - ▣ Rodar os casos de teste, analisar os resultados, identificar os mutantes equivalentes e definir novos casos de teste para matar os mutantes que não foram mortos são atividades que consomem muito tempo.
 - ▣ Mesmo em programas pequenos o número de mutantes pode ser muito grande, exigindo um tempo de execução muito alto.

Teste de Mutação

- Ferramentas de teste de mutação auxiliam o processo de testes.
- Pesquisas mostram que os testes de mutação são tão poderosos quanto os testes de fluxo de controle, testes de fluxo de dados e testes de domínio e tem sido utilizados nos testes de unidade.